Verifying Abstract Interpretations Using Multi-Language Semantics

Ling Zheng
Northeastern University

Benjamin Quiring
University of Maryland

NJPLS Dec 2025

```
let f = fun x => x in
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (f g) in
(h 3)
```

```
let f = fun x => x in
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (f g) in
(h 3)
```

which calls can apply which λ

```
let f = ℓ1fun x => x in
let add1 = ℓ2fun n => n + 1 in
let g = ℓ3fun m => 2 * c1(add1 m) in
let h = c2(f g) in
c3(h 3)
```

```
let f = ℓ1fun x => x in
let add1 = ℓ2fun n => n + 1 in
let g = ℓ3fun m => 2 * c1(add1 m) in
let h = c2(f g) in
c3(h 3)
```

```
let f = ℓ1fun x => x in
let add1 = ℓ2fun n => n + 1 in
let g = ℓ3fun m => 2 * c1(add1 m) in
let h = c2(f g) in
c3(h 3)
```

(ℓ_1, c_2)

```
let f = ℓ1fun x => x in
let add1 = ℓ2fun n => n + 1 in
let g = ℓ3fun m => 2 * c1(add1 m) in
let h = c2(f g) in
c3(h 3)
```

(ℓ_1, c_2)

```
let f = ℓ1fun x => x in
let add1 = ℓ2fun n => n + 1 in
let g = ℓ3fun m => 2 * c1(add1 m) in
let h = c2(f g) in
c3(h 3)
```

$(\ell_1, c_2), (\ell_3, c_3)$

```
let f = ℓ1fun x => x in
let add1 = ℓ2fun n => n + 1 in
let g = ℓ3fun m => 2 * c1(add1 m) in
let h = c2(f g) in
c3(h 3)
```

$(\ell_1, c_2), (\ell_3, c_3)$

```
let f = ℓ1fun x => x in
let add1 = ℓ2fun n => n + 1 in
let g = ℓ3fun m => 2 * c1(add1 m) in
let h = c2(f g) in
c3(h 3)
```

$$(\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1)$$

```
let f = ℓ1fun x => x in
let add1 = ℓ2fun n => n + 1 in
let g = ℓ3fun m => 2 * c1(add1 m) in
let h = c2(f g) in
c3(h 3)
```

$$\{(\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1)\}$$

- Which calls can apply which λ (control-flow analysis)
  $\{ (\ell_1, c_1), (\ell_1, c_2), (\ell_2, c_1), \ldots \}$

- Which calls can apply which λ (control-flow analysis)
  $\{ (\ell_1, c_1), (\ell_1, c_2), (\ell_2, c_1), \ldots \}$
- Whether an integer variable is even or odd (parity analysis)
  $\{ (\mathrm{x}, \mathrm{even}), (\mathrm{y}, \mathrm{odd}), (\mathrm{z}, \bot), (\mathrm{w}, \top), \ldots \}$

- Which calls can apply which λ (control-flow analysis)
  $\{ (\ell_1, c_1), (\ell_1, c_2), (\ell_2, c_1), \ldots \}$
- Whether an integer variable is even or odd (parity analysis)
  $\{ (\mathsf{x}, \mathsf{even}), (\mathsf{y}, \mathsf{odd}), (\mathsf{z}, \bot), (\mathsf{w}, \top), \ldots \}$
- Whether a variable is an alias of another (alias analysis)
  $\{ (\mathsf{x}, \mathsf{y}), (\mathsf{y}, \mathsf{z}), (\mathsf{z}, \mathsf{w}), \ldots \}$

- Which calls can apply which λ (control-flow analysis)
  $\{ (\ell_1, c_1), (\ell_1, c_2), (\ell_2, c_1), \ldots \}$
- Whether an integer variable is even or odd (parity analysis)
  $\{ (\mathsf{x}, \mathsf{even}), (\mathsf{y}, \mathsf{odd}), (\mathsf{z}, \bot), (\mathsf{w}, \top), \ldots \}$
- Whether a variable is an alias of another (alias analysis)
  $\{ (\mathsf{x}, \mathsf{y}), (\mathsf{y}, \mathsf{z}), (\mathsf{z}, \mathsf{w}), \ldots \}$
- Whether a type contract is always satisfied (gradual typing)
  The set of violated contracts / blames

- Which calls can apply which λ (control-flow analysis)
  $\{ (\ell_1, c_1), (\ell_1, c_2), (\ell_2, c_1), \ldots \}$
- Whether an integer variable is even or odd (parity analysis)
  $\{ (\mathsf{x}, \mathsf{even}), (\mathsf{y}, \mathsf{odd}), (\mathsf{z}, \bot), (\mathsf{w}, \top), \ldots \}$
- Whether a variable is an alias of another (alias analysis)
  $\{ (\mathsf{x}, \mathsf{y}), (\mathsf{y}, \mathsf{z}), (\mathsf{z}, \mathsf{w}), \ldots \}$
- Whether a type contract is always satisfied (gradual typing)
  The set of violated contracts / blames

Trace Events

- Instrument a standard concrete semantics

Collecting Semantics

- Instrument a standard concrete semantics

Collecting Semantics

- Instrument a standard concrete semantics
- Most precise

Collecting Semantics

- Instrument a standard concrete semantics
- Most precise
- Uncomputable

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \varsigma_3 \to \varsigma_4 \to \varsigma_5 \to \varsigma_6 \to \varsigma_7 \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \varsigma_3 \to \varsigma_4 \to \varsigma_5 \to \varsigma_6 \to \varsigma_7 \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{array}{ccccccc} \widetilde{\varsigma_a} & \longrightarrow & \widetilde{\varsigma_b} & \longrightarrow & \widetilde{\varsigma_e} & \longrightarrow & \cdots \\ & \swarrow & \uparrow & & & & \\ \widetilde{\varsigma_c} & \longrightarrow & \widetilde{\varsigma_d} & & & & \end{array}$$

Collecting semantics: Uncomputable

$$\boxed{\varsigma_1} \to \varsigma_2 \to \varsigma_3 \to \varsigma_4 \to \varsigma_5 \to \varsigma_6 \to \varsigma_7 \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\boxed{\widetilde{\varsigma_a}} \arrow[r] & \widetilde{\varsigma_b} \arrow[r] & \widetilde{\varsigma_e} \arrow[r] & \cdots \\
& \widetilde{\varsigma_c} \arrow[r] \arrow[ul] & \widetilde{\varsigma_d} &
\end{tikzcd}$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \boxed{\varsigma_2} \to \varsigma_3 \to \varsigma_4 \to \varsigma_5 \to \varsigma_6 \to \varsigma_7 \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\widetilde{\varsigma_a} \arrow[r] & \boxed{\widetilde{\varsigma_b}} \arrow[r] \arrow[d] & \widetilde{\varsigma_e} \arrow[r] & \cdots \\
& \widetilde{\varsigma_c} \arrow[r] & \widetilde{\varsigma_d} \arrow[u] &
\end{tikzcd}$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \boxed{\varsigma_3} \to \varsigma_4 \to \varsigma_5 \to \varsigma_6 \to \varsigma_7 \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\widetilde{\varsigma_a} \arrow[r] & \widetilde{\varsigma_b} \arrow[r] & \widetilde{\varsigma_e} \arrow[r] & \cdots \\
\boxed{\widetilde{\varsigma_c}} \arrow[ur] \arrow[r] & \widetilde{\varsigma_d} \arrow[u] & &
\end{tikzcd}$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \varsigma_3 \to \boxed{\varsigma_4} \to \varsigma_5 \to \varsigma_6 \to \varsigma_7 \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\widetilde{\varsigma_a} \arrow[r] \arrow[dr] & \widetilde{\varsigma_b} \arrow[r] & \widetilde{\varsigma_e} \arrow[r] & \cdots \\
\widetilde{\varsigma_c} \arrow[r] & \boxed{\widetilde{\varsigma_d}} \arrow[u] & &
\end{tikzcd}$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \varsigma_3 \to \varsigma_4 \to \boxed{\varsigma_5} \to \varsigma_6 \to \varsigma_7 \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\widetilde{\varsigma_a} \arrow[r] & \boxed{\widetilde{\varsigma_b}} \arrow[r] \arrow[d] & \widetilde{\varsigma_e} \arrow[r] & \cdots \\
& \widetilde{\varsigma_c} \arrow[r] & \widetilde{\varsigma_d} \arrow[u] &
\end{tikzcd}$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \varsigma_3 \to \varsigma_4 \to \varsigma_5 \to \boxed{\varsigma_6} \to \varsigma_7 \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\widetilde{\varsigma_a} \arrow[r] & \widetilde{\varsigma_b} \arrow[r] & \widetilde{\varsigma_e} \arrow[r] & \cdots \\
\boxed{\widetilde{\varsigma_c}} \arrow[ur] \arrow[r] & \widetilde{\varsigma_d} \arrow[u] & &
\end{tikzcd}$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \varsigma_3 \to \varsigma_4 \to \varsigma_5 \to \varsigma_6 \to \boxed{\varsigma_7} \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\widetilde{\varsigma_a} \arrow[r] & \widetilde{\varsigma_b} \arrow[r] & \widetilde{\varsigma_e} \arrow[r] & \cdots \\
& \widetilde{\varsigma_c} \arrow[r] \arrow[ul] & \boxed{\widetilde{\varsigma_d}} \arrow[u] &
\end{tikzcd}$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \varsigma_3 \to \varsigma_4 \to \varsigma_5 \to \varsigma_6 \to \varsigma_7 \to \boxed{\varsigma_8} \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\widetilde{\varsigma_a} \arrow[r] & \boxed{\widetilde{\varsigma_b}} \arrow[r] \arrow[d] & \widetilde{\varsigma_e} \arrow[r] & \cdots \\
& \widetilde{\varsigma_c} \arrow[r] & \widetilde{\varsigma_d} \arrow[u] &
\end{tikzcd}$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \varsigma_3 \to \varsigma_4 \to \varsigma_5 \to \varsigma_6 \to \varsigma_7 \to \varsigma_8 \to \boxed{\varsigma_9} \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\widetilde{\varsigma_a} \arrow[r] & \widetilde{\varsigma_b} \arrow[r] & \boxed{\widetilde{\varsigma_e}} \arrow[r] & \cdots \\
& \widetilde{\varsigma_c} \arrow[ul] \arrow[r] & \widetilde{\varsigma_d} \arrow[u] &
\end{tikzcd}$$

Collecting semantics: Uncomputable

$$\varsigma_1 \to \varsigma_2 \to \varsigma_3 \to \varsigma_4 \to \varsigma_5 \to \varsigma_6 \to \varsigma_7 \to \varsigma_8 \to \varsigma_9 \to \cdots$$

Abstract to a *finite* representation

$$\begin{tikzcd}
\widetilde{\varsigma_a} \arrow[r] & \widetilde{\varsigma_b} \arrow[r] & \widetilde{\varsigma_e} \arrow[r] & \cdots \\
& \widetilde{\varsigma_c} \arrow[ul] \arrow[r] & \widetilde{\varsigma_d} \arrow[u] &
\end{tikzcd}$$

Abstract Semantics

- Finite and computable

Abstract Semantics

- Finite and computable
- Reuse resources

Abstract Semantics

- Finite and computable
- Reuse resources
- Non-deterministic

Abstract Semantics

- Finite and computable
- Reuse resources
- Non-deterministic
- Abstract Abstract Machines (Van Horn and Might 2010)

The abstract semantics safely approximates the collecting semantics.

The abstract semantics safely approximates the collecting semantics.

All concrete program traces are covered by some abstract trace.

Separate Compilation

```
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

What can we say about `M.f` and `h`?

```
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

- `M.f` should be unknown (or •)

```
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

- `M.f` should be unknown (or •)

```
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

- `M.f` should be unknown (or •)
- Applying • should produce another •

```
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

- `M.f` should be unknown (or •)
- Applying • should produce another •

Nguyen, Tobin-Hochstadt, and Van Horn 2014 treated • this way.

```
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

- `M.f` should be unknown (or •)
- Applying • should produce another •

Nguyen, Tobin-Hochstadt, and Van Horn 2014 treated • this way.
Does • mean **any value**?

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
```

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
let f = fun x => x
```

- Link `M.f` with `id`

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
let f = fun x => x
```

- Link `M.f` with `id`
- • can represent something internally defined

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
let f = fun x => x
```

- Link `M.f` with `id`
- • can represent something internally defined
- λ bound by g has escaped!

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
let f = fun x => x
```

- Link `M.f` with `id`
- • can represent something internally defined
- λ bound by g has escaped!

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
```

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
let f = fun _ =>
 let sq = fun x => x * x in
 sq
```

- Link `M.f` with some constant function

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
let f = fun _ =>
 let sq = fun x => x * x in
 sq
```

- Link `M.f` with some constant function
- • can represent anything externally defined

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
let f = fun _ =>
 let sq = fun x => x * x in
 sq
```

- Link `M.f` with some constant function
- • can represent anything externally defined

```
(* f is defined in some module M *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

What about the λ bound by add1?

```
(* f is defined in some module M *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* f is defined in some module M *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

- Should `(M.f g)` ever return it?

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
let f = fun _ =>
 let S = fun x => x + 1 in
 S
```

```
(* f is defined in some module M *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

- `(M.f g)` should never return it

```
(* f is defined in some module M *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

- `(M.f g)` should never return it
- It should never be called when we apply `h` to 3

```
(* f is defined in some module M *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

- `(M.f g)` should never return it
- It should never be called when we apply `h` to 3
- • does **not** include internal non-escaping values

- are exactly the external values + the internal escaping values.

- Imports are •

- Imports are $\bullet$
- $[c]$ $(\bullet$ $\mathtt{v})$

- Imports are •
- [c] (• v)

  - (• v) -> •

- Imports are $\bullet$
- $[c]$ `(•  v)`
  - record trace $(\ell_\bullet, c)$
  - `(•  v) -> •`

- Imports are $\bullet$
- $[c]$ `(•  v)`
  - record trace $(\ell_\bullet, c)$
  - record `v` escaped
  - `(•  v) -> •`

- Imports are $\bullet$
- $[c]$ `(•  v)`
  - record trace $(\ell_\bullet, c)$
  - record `v` escaped
  - record trace $(\ell_v, c_\bullet)$

  - `(•  v) -> •`

- Imports are $\bullet$
- $[c]$ `(• v)`
  - record trace $(\ell_\bullet, c)$
  - record `v` escaped
  - record trace $(\ell_v, c_\bullet)$
  - `(• v) ->`
  - `(• v) -> •`

- Imports are $\bullet$
- $[c]$ `(• v)`
  - record trace $(\ell_\bullet, c)$
  - record `v` escaped
  - record trace $(\ell_v, c_\bullet)$
  - `(• v) -> (v •)`
  - `(• v) -> •`

- Imports are $\bullet$
- $[c]$ `(• v)`
  - record trace $(\ell_\bullet, c)$
  - record `v` escaped
  - record trace $(\ell_v, c_\bullet)$
  - `(• v) -> (• (v •))`
  - `(• v) -> •`

- Imports are $\bullet$
- [c] `(• v)`
  - record trace $(\ell_\bullet, c)$
  - record `v` escaped
  - record trace $(\ell_v, c_\bullet)$
  - `(• v) -> (• (v •))`
  - `(• v) -> •`

Please note that $\bullet$ is opaque

Shivers' Escape Technique 1991

- Much finer-grained
- Work with the set of escaped values instead of $\bullet$

Midtgaard, Adams, and Might 2012

- **Internal** vs **External**

Midtgaard, Adams, and Might 2012

- **Internal** vs **External**
- Galois connections

Midtgaard, Adams, and Might 2012

- Mechanizations and verified compilation

Midtgaard, Adams, and Might 2012

- Mechanizations and verified compilation
- Generalization to other trace events

Midtgaard, Adams, and Might 2012

- Simulation based soundness proof

Midtgaard, Adams, and Might 2012

- Simulation based soundness proof
- Need a generic way to differentiate the **trace events**

Multi-Language Semantics

Multi-Language Semantics

- Gradual typing

Multi-Language Semantics

- Gradual typing
- Foreign function interfaces

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
```

```
(* Main *)
let add1 = fun n => n + 1 in
let g = fun m => 2 * (add1 m) in
let h = (M.f g) in
(h 3)
```

```
(* Module M *)
let f = fun g' =>
 let sq = fun x => x * x in
 fun y => (sq y) * (g' y)
```

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

(ℓ_1, c_2)

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

(ℓ_1, c_2)

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

$(\ell_1, c_2), (\ell_3, c_3)$

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

$(\ell_1, c_2), (\ell_3, c_3)$

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

$(\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1)$

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

$$(\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1)$$

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

$(\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1), (\ell_2, c_2)$

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

$(\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1), (\ell_2, c_2)$

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

$$(\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1), (\ell_2, c_2), (\ell_1, c_1)$$

```
(* Main *)
let add1 = ℓ1fun n => n + 1 in
let g = ℓ2fun m => 2 * c1(add1 m) in
let h = c2(M.f g) in
c3(h 3)
```

```
(* Module M *)
let f = ℓ1fun g' =>
 let sq = ℓ2fun x => x * x in
ℓ3fun y => c1(sq y) * c2(g' y)
```

$$\{ (\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1), (\ell_2, c_2), (\ell_1, c_1) \}$$

$$\{ (\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1), (\ell_2, c_2), (\ell_1, c_1) \}$$

$$\{ (\ell_\bullet, c_2), (\ell_\bullet, c_3), \qquad , (\ell_2, c_\bullet), \qquad \}$$

- Boundary

$$\{\ (\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1), (\ell_2, c_2), (\ell_1, c_1)\ \}$$

$$\{\ (\ell_\bullet, c_2), (\ell_\bullet, c_3), (\ell_\bullet, c_\bullet), (\ell_2, c_\bullet), \qquad\qquad \}$$

- Boundary
- External

$$\{ (\ell_1, c_2), (\ell_3, c_3), (\ell_2, c_1), (\ell_2, c_2), (\ell_1, c_1) \}$$

$$\{ (\ell_\bullet, c_2), (\ell_\bullet, c_3), (\ell_\bullet, c_\bullet), (\ell_2, c_\bullet), (\ell_1, c_1) \}$$

- Boundary
- External
- Internal

- Mechanizable simulation-based soundness proofs
- Generalizable to other trace events

zheng.lingx@northeastern.edu
bquiring@umd.edu